

E06 — Evaluate Reverse Polish Notation (Stack)

Experiment: 6 | LeetCode: #150 | CO: CO2, CO3 | BT Level: BT5

Problem Statement

You are given an array of strings `tokens` that represents an arithmetic expression in **Reverse Polish Notation (RPN)** (also called postfix notation).

Evaluate the expression and return an integer representing the value.

Rules:

- Valid operators are `+`, `-`, `*`, and `/`.
- Each operand may be an integer or another expression.
- Division between two integers should **truncate toward zero**.
- The given RPN expression is always valid.

Example 1:

Input: `tokens = ["2", "1", "+", "3", "*"]`

Output: 9

Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: `tokens = ["4", "13", "5", "/", "+"]`

Output: 6

Explanation: $(4 + (13 / 5)) = 4 + 2 = 6$

Example 3:

Input: `tokens = ["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "`

Output: 22

Concept Review: RPN Evaluation with Stack

RPN eliminates the need for parentheses by specifying operator order through token position.

Algorithm:

```
for each token:
    if token is a number:
        push onto stack
    else (token is operator):
        pop two operands: b = pop(), a = pop()
        apply: a op b
```

```
    push result onto stack
```

```
return stack[0]
```

Important: `b = pop()` first (top of stack = right operand), then `a = pop()` (left operand). Order matters for `-` and `/`.

Implementation

```
def evalRPN(tokens):
    """
    Evaluate Reverse Polish Notation using a stack.
    Time: O(n) | Space: O(n)
    n = number of tokens
    """
    stack = []
    operators = {'+', '-', '*', '/'}

    for token in tokens:
        if token in operators:
            b = stack.pop() # Right operand (top of stack)
            a = stack.pop() # Left operand

            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                # Truncate toward zero (not floor division)
                stack.append(int(a / b))
        else:
            stack.append(int(token)) # Push integer

    return stack[0]
```

Python Division Note

```
# Python's // is floor division (rounds toward -infinity):
7 // 2 = 3      (correct, same as truncate)
-7 // 2 = -4    (WRONG - truncate should give -3)

# Correct: use int(a / b) which truncates toward zero:
int(-7 / 2) = int(-3.5) = -3    ✓
```

Detailed Trace

Example 1: `["2", "1", "+", "3", "*"]`

Token	Action	Stack
"2"	push 2	[2]
"1"	push 1	[2, 1]
"+"	b = 1, a = 2; push $2 + 1 = 3$	[3]
"3"	push 3	[3, 3]
"*"	b = 3, a = 3; push $3 * 3 = 9$	[9]

Result: 9 ✓

Example 2: ["4", "13", "5", "/", "+"]

Token	Action	Stack
"4"	push 4	[4]
"13"	push 13	[4, 13]
"5"	push 5	[4, 13, 5]
"/"	b = 5, a = 13; push $\text{int}(13/5) = \text{int}(2.6) = 2$	[4, 2]
"+"	b = 2, a = 4; push $4 + 2 = 6$	[6]

Result: 6 ✓

Example 3 (Complex):

["10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"]

Token	Action	Stack
"10"	push	[10]
"6"	push	[10, 6]
"9"	push	[10, 6, 9]
"3"	push	[10, 6, 9, 3]
"+"	b = 3, a = 9; $9 + 3 = 12$	[10, 6, 12]
"-11"	push	[10, 6, 12, -11]
"*"	b = -11, a = 12; $12 \times (-11) = -132$	[10, 6, -132]
"/"	b = -132, a = 6; $\text{int}(6/-132) = \text{int}(-0.045) = 0$	[10, 0]
"*"	b = 0, a = 10; $10 \times 0 = 0$	[0]
"17"	push	[0, 17]
"+"	b = 17, a = 0; $0 + 17 = 17$	[17]
"5"	push	[17, 5]
"+"	b = 5, a = 17; $17 + 5 = 22$	[22]

Result: 22 ✓

Edge Cases

```
# Negative numbers (valid tokens: "-3", "-11")
["3", "-11", "*"]    # 3 × (-11) = -33

# Division truncates toward zero
["-10", "3", "/"]    # int(-10/3) = int(-3.33) = -3 (NOT -4)
["10", "-3", "/"]    # int(10/-3) = int(-3.33) = -3 (NOT -4)

# Single element
["18"]    # → 18
```

Test Suite

```
def test_eval_rpn():
    test_cases = [
        ("2", "1", "+", "3", "*"), 9),
        ("4", "13", "5", "/", "+"), 6),
        ("10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"], 22)
        ("18"], 18),
        (["-10", "3", "/"], -3),    # Truncation test
        ("3", "-4", "*"], -12),
        ("5", "1", "2", "+", "4", "*", "+", "3", "-"], 14),    # 5+1+2=3, 3*4=12
    ]

    for tokens, expected in test_cases:
        result = evalRPN(tokens)
        status = "PASS" if result == expected else "FAIL"
        print(f"{status}: evalRPN({tokens}) = {result} (expected {expected})")

test_eval_rpn()
```

Complexity Analysis

Metric	Value
Time	$O(n)$ — single pass through tokens
Space	$O(n)$ — stack holds at most $(n+1)/2$ operands

Key Takeaways

- **Stack** is the natural data structure for expression evaluation — operands pushed, popped on operator.
 - **Pop order matters:** `b = pop()` is the right operand, `a = pop()` is the left operand.
 - **Division truncation:** use `int(a / b)`, not `a // b` (Python's floor division differs from truncation for negative numbers).
 - This is the evaluation step for any postfix expression (complements the infix → postfix conversion in L25).
-

References

- LeetCode #150 — Evaluate Reverse Polish Notation
- Cormen et al., *Introduction to Algorithms*, Ch. 10.1 (Stacks)
- L24.md (Evaluation of Postfix Expressions) — full postfix theory
- L25.md (Infix to Postfix Conversion) — Shunting-Yard algorithm